

futures__emsx__strategy

API & Component Contracts

Reference documentation

Contents

1	Overview	2
1.1	Architectural contract	2
1.2	Event types (canonical)	2
1.3	Notation used in this document	2
2	Module core/	3
2.1	core.clock	3
2.2	core.enums	3
2.3	core.exceptions	4
2.4	core.identifiers	4
2.5	core.logging	5
2.6	core.events	5
3	Module config/	7
4	Module market_data/	8
4.1	market_data.base	8
4.2	market_data.bloomberg_provider	8
4.3	market_data.mock_provider	8
4.4	market_data.bar_builder	9
4.5	market_data.tick_store	9
4.6	market_data.contract_resolver	9
4.7	market_data.stale_data_monitor	9
5	Module strategy/	11
5.1	strategy.base	11
5.2	strategy.indicators	11
5.3	strategy.signal_state	11
5.4	strategy.target_position	11
5.5	strategy.minute_strategy	12
5.6	strategy.roll_logic	12

6	Module portfolio/	13
6.1	portfolio.positions	13
6.2	portfolio.fills	13
6.3	portfolio.pnl	13
6.4	portfolio.exposure	13
6.5	portfolio.reconciliation	14
7	Module orders/	15
7.1	orders.models	15
7.2	orders.idempotency	15
7.3	orders.throttles	15
7.4	orders.sizing	15
7.5	orders.lifecycle	16
7.6	orders.order_manager	16
8	Module risk/	17
8.1	risk.limits	17
8.2	risk.validations	17
8.3	risk.session_checks	17
8.4	risk.kill_switch	17
8.5	risk.pre_trade	17
9	Module execution/	19
9.1	execution.base	19
9.2	execution.emsx_mapper	19
9.3	execution.emsx_requests	19
9.4	execution.emsx_subscriptions	20
9.5	execution.emsx_state_machine	20
9.6	execution.emsx_adapter	20
9.7	execution.paper_adapter	21
10	Module storage/	22
10.1	storage.db	22
10.2	storage.event_log	22
10.3	storage.repositories	22
10.4	storage.idempotency_backend	22
10.5	storage.snapshots	23
11	Module messaging/	24
11.1	messaging.bus	24
11.2	messaging.codec	24
11.3	messaging.in_memory_bus	24
11.4	messaging.kafka_bus	25
11.5	messaging.redis_bus	25

12 Module monitoring/	26
12.1 monitoring.metrics	26
12.2 monitoring.health	26
12.3 monitoring.alerts	26
13 Module backtest/	27
13.1 backtest.slippage	27
13.2 backtest.commission	27
13.3 backtest.replay	27
13.4 backtest.engine	27
13.5 backtest.reports	27
14 Module app/	29
14.1 app.topics	29
14.2 app.wiring	29
14.3 app.run_market_data	29
14.4 app.run_strategy	30
14.5 app.run_execution	30
14.6 app.run_reconciler	30
14.7 app.run_all_local	30
15 Topology & guarantees	32
15.1 Service boundaries	32
15.2 Idempotency & at-least-once delivery	32
15.3 State-machine invariants	32
15.4 Phased rollout flags	32
15.5 Error budget for live trading	32
16 Quick reference — minimum contracts to satisfy when extending	33
16.1 Adding a market-data vendor	33
16.2 Adding a venue	33
16.3 Adding a strategy	33
16.4 Adding a risk check	33
16.5 Adding a bus implementation	33
17 Test contract surface	34

1 Overview

1.1 Architectural contract

The system is partitioned so each layer depends only on the layer above it on the event flow, never the other way:

```
Bloomberg market data
--> Market Data Service      (ticks, bars)
--> Strategy                  (TargetPosition)
--> Order Manager             (OrderIntent)
--> Pre-Trade Risk Gateway    (RiskDecision)
--> Execution Adapter         (ExecutionAck)
--> EMSX                      (route updates, fills)
--> Portfolio & Reconciler    (PositionBook, FillLedger, PnL)
```

Hard rules:

- Strategies emit only **TargetPosition**. They never construct orders, never call EMSX, never know about routes.
- Order intents pass through **PreTradeRiskGateway** before reaching any **ExecutionAdapter**.
- Execution adapters are the only modules that import **blpapi**. Other layers work without it.
- Every **OrderIntent** has a deterministic **idempotency_key** (SHA-1 of strategy + instrument + bar-end + side + qty), so replays and restarts cannot duplicate orders.

1.2 Event types (canonical)

Event	Producer	Consumer(s)
MarketTick	Market data service	Bar builder, tick store, stale monitor
BarClosed	Bar builder	Strategy
TargetPosition	Strategy	Order manager
OrderIntent	Order manager	Risk gateway
RiskDecision	Risk gateway	Execution service
ExecutionAck	Execution adapter (sync)	Order lifecycle
ExecutionUpdate	Execution adapter (async subs)	Lifecycle, working book, monitoring
FillUpdate	Execution adapter (async subs)	Position book, PnL, fill ledger
KillSwitchEvent	Kill switch	All services (refuse to trade)

1.3 Notation used in this document

Each entry contains:

- Purpose** one-line summary.
- Signature** full Python signature.
- Pre** caller-side preconditions.
- Post** observable effects after a successful call.
- Raises** exceptions raised.

When an item has no preconditions worth stating, the **Pre** field is omitted.

2 Module core/

Foundational primitives: clocks, canonical events, enums, exceptions, identifiers, logging.

2.1 core.clock

Clock (abstract)

Purpose	Time source. Decouples wall-clock from simulation/replay.
Signature	<code>class Clock(ABC): def now(self) -> datetime</code>
Post	Returns timezone-aware UTC <code>datetime</code> .

SystemClock

Purpose	Wall-clock implementation.
Signature	<code>class SystemClock(Clock): def now(self) -> datetime</code>
Post	<code>now()</code> returns <code>datetime.now(timezone.utc)</code> .

FixedClock

Purpose	Returns a single fixed time. Used in deterministic unit tests.
Signature	<code>class FixedClock(Clock): __init__(fixed: datetime) ; set(t: datetime) -> None</code>
Pre	<code>fixed</code> must be tz-aware.

SimulatedClock

Purpose	Monotonic clock for backtests.
Signature	<code>class SimulatedClock(Clock): __init__(start: datetime) ; advance_to(t: datetime) -> None</code>
Pre	<code>advance_to(t)</code> requires $t \geq$ current internal time.
Post	Internal time is set to <code>t</code> .
Raises	<code>ValueError</code> if <code>t</code> would rewind time.

2.2 core.enums

Side (BUY, SELL)

Purpose	Order side; carries arithmetic sign for position math.
Signature	<code>side.sign -> int</code> # +1 for BUY, -1 for SELL
Signature	<code>Side.from_delta(delta: int) -> Side</code>
Raises	<code>ValueError</code> when <code>delta == 0</code> .

OrderType

Members	MKT, LMT, STP, STP_LMT.
----------------	-------------------------

TimeInForce

Members	DAY, GTC, IOC, FOK, GTD.
----------------	--------------------------

OrderStatus

Purpose	Venue-neutral lifecycle status, mapped from EMSX route status.
Members	NEW, RISK_APPROVED, RISK_REJECTED, SENT, ACCEPTED, WORKING, PART_FILLED, FILLED, CANCEL_PENDING, CANCELLED, MODIFY_PENDING, REJECTED, ROUTE_ERR, UNKNOWN.
Const	TERMINAL_ORDER_STATUSES = \{FILLED, CANCELLED, REJECTED, ROUTE_ERR\}.

RouteStatus, KillSwitchState

RouteStatus	Mirror of EMSX route status strings: SENT, WORKING, PARTFILLED, FILLED, CANCEL, REJECTED, ROUTE-ERR.
KillSwitchState	ARMED (trading allowed) or TRIPPED (all orders rejected).

2.3 core.exceptions

All custom exceptions inherit **FESError**. Layer-specific subclasses:

ConfigError	invalid or missing config.
MarketDataError	market-data session/subscription failure.
StaleMarketDataError	subclass for staleness specifically.
StrategyError	strategy invariant violation.
OrderError	bad order intent.
DuplicateOrderError	re-submission of a known idempotency key.
RiskRejection	raised when a hard check fails; <code>.reasons: list[str]</code> .
ExecutionError	generic venue error.
EMSXError	EMSX-specific.
ReconciliationBreak	internal vs. external positions disagree.

2.4 core.identifiers

new_uuid, correlation_id

Signature	<code>new_uuid() -> str</code>
Signature	<code>correlation_id(prefix: str = "corr") -> str</code>
Post	Returns globally unique strings.

idempotency_key

Purpose	Deterministic key used by OrderManager to dedupe submissions.
Signature	<code>idempotency_key(strategy_id: str, instrument: str, bar_end: datetime, side: str, qty: int) -> str</code>
Post	Same inputs $\Rightarrow$ same key (SHA-1, truncated). Replays produce identical keys.

emsx_request_id

Signature `emsx_request_id(order_id: str, action: str) -> str`
Post Unique correlation tag for an EMSX request.

2.5 core.logging

configure_logging, get_logger

Signature `configure_logging(level: str = "INFO", json: bool = True) -> None`
Signature `get_logger(name: str) -> structlog.BoundLogger`
Post All logs are structured (key/value), ISO-8601 UTC timestamped, level-filtered.

2.6 core.events

All events are frozen @dataclass(slots=True, frozen=True); immutable across the bus.

MarketTick

Fields `instrument, bid?, ask?, last?, volume?, exchange_timestamp?, receive_timestamp.`
Note ? = optional. `receive_timestamp` is always set by the provider.

BarClosed

Fields `instrument, start_time, end_time, open, high, low, close, volume,`
 `interval_minutes=1.`
Invariant `low <= open, close <= high; end_time > start_time.`

TargetPosition

Fields `strategy_id, instrument, target_qty: int, timestamp, reason, metadata.`
Invariant `target_qty` is signed; positive means net long.

OrderIntent

Fields `strategy_id, instrument, side, qty (>0), order_type, time_in_force,`
 `idempotency_key, source_timestamp, limit_price?, stop_price?, metadata.`
Invariant `qty > 0`. Direction lives in `side`, not the sign of `qty`.

RiskDecision

Fields `order_key, approved: bool, reasons: tuple[str,...], decided_at.`
Factories `RiskDecision.approve(order_key, decided_at),` `RiskDecision.reject(order_key,`
 `reasons, decided_at).`

ExecutionAck, ExecutionUpdate, FillUpdate, KillSwitchEvent

ExecutionAck	Synchronous response of submit/cancel/modify: order_id, route_id?, venue_request_id?, accepted: bool, message, timestamp.
ExecutionUpdate	Async push from venue subscription: order_id, route_id?, instrument, status (OrderStatus), filled_qty, avg_price?, leaves_qty, timestamp, raw.
FillUpdate	A trade execution: order_id, route_id?, instrument, side, fill_qty, fill_price, timestamp.
KillSwitchEvent	tripped, reason, timestamp, actor.

3 Module config/

Strongly typed pydantic models loaded from six YAML files under config/.

load_yaml, load_app_config

Signature	load_yaml(path: str Path) -> dict[str, Any]
Signature	load_app_config(config_dir: str Path) -> AppConfig
Pre	config_dir contains all six YAMLs (instruments, strategies, risk_limits, emsx, market_data, environments).
Post	Returns validated AppConfig ; raises ConfigError on missing/invalid files.

InstrumentConfig

Fields	symbol, bloomberg_topic, exchange, currency, tick_size, point_value, min_qty, max_qty, roll_days_before_expiry, session_tz, session_open, session_close.
Invariant	tick_size, point_value > 0; 1 <= min_qty <= max_qty.

InstrumentsConfig

Signature	by_symbol(symbol: str) -> InstrumentConfig
Raises	ConfigError if symbol not configured.

StrategyConfig, StrategiesConfig

StrategyConfig	strategy_id, type, instrument, interval_minutes=1, base_qty=1, params: dict.
StrategiesConfig	holds strategies: list[StrategyConfig].

RiskLimitsConfig

Fields	max_order_qty, max_position, max_notional, max_orders_per_minute=10, max_cancels_per_minute=20, stale_data_seconds=30, require_market_session=true, kill_switch_armed=true.
---------------	---

EMSXConfig

Fields	host="localhost", port=8194, service="//blp/emapisvc_beta", use_server_api=True, trader_uuid?, account?, broker="BMTB", handling_instruction="ANY", strategy="VWAP", default_order_type="MKT", default_tif="DAY".
Invariant	service must start with //blp/.

MarketDataConfig, EnvironmentsConfig, AppConfig

MarketDataConfig	provider, host, port, service, fields, historical_service, bar_interval_minutes.
EnvironmentsConfig	name, paper_trading, db_url, event_log_path, bus, bus_url?, metrics_port.
AppConfig	aggregate of all six configs.

4 Module market_data/

4.1 market_data.base

MarketDataProvider (abstract)

Purpose	The single seam that decouples the strategy from Bloomberg. All implementations conform to this interface.
Signature	<pre>class MarketDataProvider(ABC): def start(self) -> None def stop(self) -> None def subscribe(self, instruments: list[str], fields: list[str]) -> None def on_tick(self, callback: Callable[[MarketTick], None]) -> None def request_historicalBars(self, instrument, start, end, interval_minutes=1) -> Iterable[BarClosed]</pre>
Contract	• <code>start</code> initializes the session. Must be called before <code>subscribe</code>. • <code>subscribe</code> is idempotent: re-subscribing the same instrument has no effect. • <code>on_tick</code> callbacks are invoked once per <code>MarketTick</code> in arrival order; exceptions in callbacks must not kill the provider thread. • <code>stop</code> releases the session; further calls are no-ops.

4.2 market_data.bloomberg_provider

BloombergMarketDataProvider

Purpose	BLPAPI implementation of <code>MarketDataProvider</code> . Threaded event loop pulls events from the session and dispatches <code>SUBSCRIPTION_DATA</code> events to callbacks.
Signature	<pre>__init__(host="localhost", port=8194, service="//blp/mktdata", historical_service="//blp/refdata", topic_to_symbol: dict[str, str] None = None)</pre>
topic_to_symbol	Optional translation map applied on receive: ticks subscribed under a Bloomberg topic (e.g. <code>ESM6 Index</code>) are reported under the application's internal symbol (e.g. <code>ES1</code>). When omitted, the topic is used verbatim.
Wiring	<code>app.run_market_data</code> constructs the provider from <code>market_data.yaml</code> (host, port, service, historical_service) and subscribes to <code>instruments.yaml::bloomberg_topic</code> ; the translation map is built from <code>\{i.bloomberg_topic: i.symbol\}</code> .
Pre	<code>blpapi</code> is installed; reachable Bloomberg session (Desktop API, SAPI, or B-PIPE).
Raises	<code>MarketDataError</code> if session start or service open fails, or if <code>blpapi</code> is missing.
Fields supported	<code>BID</code> , <code>ASK</code> , <code>LAST_PRICE</code> , <code>VOLUME</code> , <code>EVT_TIME_STAMP</code> (any subset).

4.3 market_data.mock_provider

MockMarketDataProvider

Purpose	Test and paper-trading replacement; ticks are pushed manually.
Signature	<code>push_tick(tick: MarketTick) -> None</code>
Signature	<code>MockMarketDataProvider.tick(instrument, last, ts=None, volume=1) -> MarketTick</code>
Post	Synchronously delivers the tick to every registered callback.

4.4 market_data.bar_builder

MinuteBarBuilder

Purpose	Streaming OHLCV aggregator. Closes the current bar deterministically when a tick crosses into the next minute (or <code>flush()</code> is called).
Signature	<pre>__init__(interval_minutes: int = 1) on_tick(tick: MarketTick) -> None on_bar(callback: Callable[[BarClosed], None]) -> None flush(instrument: str None = None) -> None</pre>
Contract	• <code>on_tick</code> uses <code>exchange_timestamp</code> when present, else <code>receive_timestamp</code>. • Price source order: <code>last</code> → mid of (<code>bid</code>, <code>ask</code>); tick is dropped if neither is available. • Bars emitted only when a strictly later minute arrives. Use <code>flush</code> at shutdown to emit pending bars.
Raises	<code>ValueError</code> if <code>interval_minutes <= 0</code> .

4.5 market_data.tick_store

InMemoryTickStore

Purpose	Bounded per-instrument ring buffer of recent ticks (default 1024).
Signature	<pre>append(tick: MarketTick) -> None last(instrument: str) -> MarketTick None recent(instrument: str, n: int = 100) -> list[MarketTick] instruments() -> list[str]</pre>
Concurrency	All operations are thread-safe.

4.6 market_data.contract_resolver

ContractResolver

Purpose	Resolves logical symbols to active contracts and decides when to roll.
Signature	<pre>resolve(symbol: str, as_of: date None = None) -> InstrumentConfig should_roll(ins: InstrumentConfig, as_of: date, expiry: date) -> bool known_symbols() -> list[str] validate(symbol: str) -> None # raises ConfigError if unknown</pre>

4.7 market_data.stale_data_monitor

StaleDataMonitor

Purpose Tracks the freshness of incoming ticks; consumed by `PreTradeRiskGateway`.

Signature

```
__init__(max_age_seconds: int, clock: Clock | None = None)
on_tick(tick: MarketTick) -> None
last_seen(instrument: str) -> datetime | None
is_stale(instrument: str) -> bool
age_seconds(instrument: str) -> float | None
```

Contract

- An instrument that has never been seen is stale.
 - Naive timestamps are treated as UTC for the age computation.
-

5 Module strategy/

5.1 strategy.base

PortfolioView (Protocol)

Purpose Read-only window the strategy is allowed to see.

Signature

```
class PortfolioView(Protocol):
    def position(self, instrument: str) -> int
    def avg_cost(self, instrument: str) -> float | None
```

Strategy (abstract)

Purpose Consumes bars, emits target positions. Pure logic; no I/O.

Signature

```
class Strategy(ABC):
    strategy_id: str
    def on_bar(self, bar: BarClosed, portfolio: PortfolioView) -> list[TargetPosition]
    def warmup(self, bars: list[BarClosed]) -> None # optional
```

Contract

- `on_bar` must be pure with respect to external state apart from the strategy's own indicator state.
- Targets returned must reference `bar.instrument` or any instrument the strategy declared at init.

5.2 strategy.indicators

SMA, EMA, RollingZ

SMA `__init__(period: int); update(x: float) -> float | None; ready: bool.` Returns None until period observations have been seen.

EMA Same shape; smoothing factor $\alpha = 2/(period + 1)$.

RollingZ `__init__(period >= 2);` returns 0.0 when sample stdev is zero, else $(x - \mu)/\sigma$.

Raises `ValueError` for non-positive periods.

5.3 strategy.signal_state

SignalState

Purpose Bundles fast/slow EMA indicators into one object.

Signature

```
SignalState(fast_lookback: int, slow_lookback: int)
update(close: float) -> float | None # fast - slow, None until both ready
ready: bool
```

5.4 strategy.target_position

build_target_position

Purpose	Pure function: <code>signal</code> → <code>TargetPosition</code> .
Signature	<pre>build_target_position(strategy_id: str, instrument: str, signal: float, timestamp: datetime, base_qty: int, entry_threshold: float = 0.0, exit_threshold: float = 0.0, max_position: int None = None,) -> TargetPosition</pre>
Behaviour	<code>signal > entry_threshold</code> ⇒ <code>+base_qty</code> ; <code>signal < -entry_threshold</code> ⇒ <code>-base_qty</code> ; otherwise 0. Clipped to <code>[-max_position, +max_position]</code> if given.

5.5 strategy.minute_strategy

MinuteFuturesStrategy

Purpose	Reference EMA-crossover strategy on minute bars.
Signature	<pre>MinuteFuturesStrategy(strategy_id: str, instrument: str, base_qty: int, params: dict None = None)</pre>
Params keys	<code>fast_lookback</code> (default 5), <code>slow_lookback</code> (default 20), <code>entry_threshold</code> , <code>exit_threshold</code> , <code>max_position_contracts</code> .
Post	Each <code>on_bar</code> call returns either <code>[]</code> (warmup) or one <code>TargetPosition</code> for <code>self.instrument</code> .

5.6 strategy.roll_logic

RollLogic

Signature	<pre>RollLogic(instrument: InstrumentConfig) evaluate(active_contract: str, next_contract: str, active_expiry: date, as_of: date) -> RollDecision</pre>
RollDecision	<code>should_roll</code> , <code>from_contract</code> , <code>to_contract?</code> , <code>reason</code> .
Rule	Roll when <code>(expiry - as_of).days <= roll_days_before_expiry</code> .

6 Module portfolio/

6.1 portfolio.positions

PositionBook

Purpose Authoritative in-memory net position + weighted-average cost per instrument. Updated by fills.

Signature

```
position(instrument: str) -> int
avg_cost(instrument: str) -> float | None
apply_fill(fill: FillUpdate) -> None
snapshot() -> dict[str, tuple[int, float]] # symbol -> (qty, avg_cost)
set(instrument: str, qty: int, avg_cost: float = 0.0) -> None
is_flat(instrument: str) -> bool
```

Avg-cost rule Let q_{prior} be the position before the fill, q' the signed fill quantity, and $q_{\text{new}} = q_{\text{prior}} + q'$.

- **Open from flat / stack same side:** avg becomes the qty-weighted average of prior basis and fill price.
- **Partial close** (q_{prior} and q_{new} same non-zero sign): avg *unchanged*.
- **Full close** ($q_{\text{new}} = 0$): avg reset to 0.
- **Flip** (q_{prior} and q_{new} opposite non-zero signs): new avg = fill price.

Concurrency Thread-safe (internal lock).

6.2 portfolio.fills

FillLedger

Purpose Append-only fill record, indexed by order_id.

Signature

```
record(fill: FillUpdate) -> None
for_order(order_id: str) -> list[FillUpdate]
all() -> list[FillUpdate]
total_filled(order_id: str) -> int
```

6.3 portfolio.pnl

PnLCalculator

Purpose Realized PnL on closing trades + unrealized PnL against a mark map. Uses each instrument's point_value.

Signature

```
__init__(instruments: InstrumentsConfig)
apply_fill(fill: FillUpdate) -> None
realized() -> float
unrealized(marks: dict[str, float]) -> float
snapshot(marks: dict[str, float]) -> PnLSnapshot # (realized, unrealized, total)
```

Math On closing n contracts of a long at avg $\bar{p}$ at price p : realized $+= n \cdot (p - \bar{p}) \cdot \text{pointValue}$.

6.4 portfolio.exposure

ExposureCalculator

Signature

```
notional(instrument: str, qty: int, price: float) -> float
gross_notional(positions: PositionBook, marks: dict[str, float]) -> float
per_instrument(positions, marks) -> list[Exposure]
```

Formula

$\text{notional} = |qty| \cdot \text{price} \cdot \text{pointValue}.$

6.5 portfolio.reconciliation

Reconciler**Signature**

```
compare(external_positions: dict[str, int]) -> list[PositionBreak]
force_resync(external_positions: dict[str, int]) -> None
```

PositionBreak instrument, internal_qty, external_qty; computed delta = external - internal.

Note force_resync is destructive and trusts the external source — requires explicit operator approval.

7 Module orders/

7.1 orders.models

WorkingOrderBook

Purpose	Net signed working quantity per instrument across all open orders. Consumed by OrderManager to avoid double-sending.
Signature	<pre> upsert(order_id, instrument, side, leaves_qty, status) -> None remove(order_id: str) -> None net_working_qty(instrument: str) -> int # signed snapshot() -> list[tuple[order_id, sym, Side, qty, OrderStatus]] </pre>
Auto-removal	Terminal statuses (FILLED, CANCELLED, REJECTED, ROUTE_ERR) or leaves_qty<=0 remove the row.

7.2 orders.idempotency

IdempotencyBackend (protocol), IdempotencyStore

IdempotencyBackend protocol

```

has(key: str) -> bool
add(key: str) -> None
claim(key: str) -> bool    # ATOMIC: True iff this caller inserted the row

seen(key: str) -> bool
mark(key: str) -> None
claim(key: str) -> bool

```

IdempotencyStore claim must be race-safe across concurrent callers (and processes, when the backend is durable). **SqliteIdempotencyBackend** uses “INSERT OR IGNORE” plus `cursor.rowcount`; the in-memory backend uses an internal lock. Check-then-add is not sufficient.

Backends `_InMemoryBackend` (default, in-process) and **SqliteIdempotencyBackend** (persistent, race-safe).

7.3 orders.throttles

RateLimiter

Purpose	Sliding-window event limiter (e.g. orders/minute).
Signature	<pre> __init__(max_events: int, window_seconds: int, clock: Clock None = None) try_acquire() -> bool # True if event accepted current() -> int # events currently in window </pre>

7.4 orders.sizing

slice_order

Purpose	Break a large parent qty into child clips.
Signature	<code>slice_order(total_qty: int, max_clip: int) -> list[int]</code>
Post	<code>sum(result) == total_qty</code> ; all(<code>c <= max_clip</code>); returns [] when <code>total_qty <= 0</code> .
Raises	<code>ValueError</code> when <code>max_clip <= 0</code> .

7.5 orders.lifecycle

OrderRecord, OrderLifecycle

OrderRecord `order_id, strategy_id, instrument, side, qty, idempotency_key, created_at, status, venue_order_id?, route_id?, filled_qty=0, avg_price?, last_update?, reasons[]`.

OrderLifecycle

```

register(rec: OrderRecord) -> None
get(order_id: str) -> OrderRecord | None
by_venue_id(venue_order_id: str) -> OrderRecord | None
resolve(order_id: str) -> OrderRecord | None
update_status(order_id, status, filled_qty?, avg_price?,
               venue_order_id?, route_id?, timestamp?) -> OrderRecord | None
set_venue_info(order_id, venue_order_id?, route_id?) -> OrderRecord | None
all() -> list[OrderRecord]

```

resolve Looks up by internal id first, then by venue id; lets a single handler work for both the paper adapter (which reports the internal id in callbacks) and EMSX (which reports EMSX_SEQUENCE).

set_venue_info Updates venue identifiers only; never overwrites **status** or fill data, so a callback that has already advanced the order is not regressed by a post-submit bookkeeping write.

7.6 orders.order_manager

OrderManager

Purpose Pure deduplicating mapping `TargetPosition` $\rightarrow$ list of `OrderIntent`.

Signature

```

__init__(positions: PositionBook, working: WorkingOrderBook,
          idempotency: IdempotencyStore,
          default_order_type: OrderType = OrderType.MKT,
          default_tif: TimeInForce = TimeInForce.DAY,
          max_clip: int | None = None)

on_target(target: TargetPosition) -> list[OrderIntent]

```

Algorithm

1. $\Delta = \text{target.target_qty} - \text{position} - \text{net working qty}$.
2. If $\Delta = 0$, return `[]`.
3. Otherwise `side = Side.from_delta( $\Delta$ )`, `qty =  $|\Delta|$` .
4. Optionally slice with `max_clip`.
5. For each clip, compute a deterministic `idempotency_key`. If `idempotency.seen(key)` is true (a previous submission was committed) skip; otherwise emit the intent. The manager does NOT claim the key here — claim is the runner's responsibility, post risk-approval.

Why seen, not claim?

A transient risk rejection (stale data, session closed) would otherwise permanently poison the key and block retries.

8 Module risk/

8.1 risk.limits

RiskLimits

Purpose	Frozen dataclass mirror of RiskLimitsConfig used by the gateway.
Fields	max_order_qty, max_position, max_notional, stale_data_seconds, require_market_session, max_orders_per_minute, max_cancels_per_minute.

8.2 risk.validations

ValidationCheck

Purpose	Reusable pass/fail check result.
Signature	ValidationCheck.ok(name) / ValidationCheck.fail(name, reason).
Fields	name, passed: bool, reason: str.

8.3 risk.session_checks

SessionChecker

Signature	is_open(now_utc: datetime) -> bool
Contract	Handles wrap-around sessions where close < open (overnight). Uses the instrument's session_tz .

8.4 risk.kill_switch

KillSwitch

Signature	<pre> __init__(armed: bool = True, clock: Clock None = None) is_tripped -> bool # property trip(reason: str, actor: str = "system") -> KillSwitchEvent arm(actor: str = "operator") -> KillSwitchEvent reason -> str None # property </pre>
Contract	When tripped, the pre-trade gateway rejects every order. Tripping is logged at error level; arming at warning .

8.5 risk.pre_trade

PreTradeRiskGateway

Purpose Mandatory gate. Every **OrderIntent** must pass before reaching **ExecutionAdapter**.

Signature

```
__init__(
    limits: RiskLimits,
    instruments: InstrumentsConfig,
    positions: PositionBook,
    exposure: ExposureCalculator,
    stale_monitor: StaleDataMonitor,
    kill_switch: KillSwitch,
    order_rate: RateLimiter,
    get_mark: Callable[[str], float | None] | None = None,
    clock: Clock | None = None,
)

validate(intent: OrderIntent) -> CheckResult
```

Checks (all must pass) • **kill_switch** — rejects if tripped.

- **max_order_qty** — $1 \leq \text{qty} \leq \text{limit}$.
- **max_position** — projected $|\text{pos}|$ after fill $\leq \text{limit}$.
- **max_notional** — $|\text{qty}| \cdot \text{mark} \cdot \text{pointValue} \leq \text{limit}$. Skipped when mark is unknown.
- **market_session** — skipped if **require_market_session** is false.
- **stale_market_data** — rejects if last tick older than **stale_data_seconds**.
- **order_rate_limit** — consumes a slot from the **order_rate** limiter.

Rate-limit ordering

order_rate_limit is the only check with a side effect (token consumption). It runs *only* when every other check has already passed, so a transient stale-data or session rejection cannot drain the budget.

CheckResult **decision**: RiskDecision, **checks**: tuple[ValidationCheck,...].

9 Module execution/

9.1 execution.base

ExecutionAdapter (abstract)

Purpose Single interface for any venue (EMSX, FIX, paper).

Signature

```
class ExecutionAdapter(ABC):
    def start(self) -> None
    def stop(self) -> None
    def submit_order(self, order: OrderIntent) -> ExecutionAck
    def cancel_order(self, order_id: str) -> ExecutionAck
    def modify_order(self, order_id: str, changes: dict) -> ExecutionAck
    def on_execution_update(self, callback: ExecutionUpdateCallback) -> None
    def on_fill(self, callback: FillCallback) -> None
```

Contract

- Sync calls (submit/cancel/modify) return an **ExecutionAck** immediately; persistent state is delivered asynchronously via callbacks.
- Multiple callbacks may be registered for the same event type; all are invoked.

9.2 execution.emsx_mapper

EMSXMapper

Purpose Only place that knows EMSX wire field names. Translates both directions.

Signature

```
to_create_order_and_route(intent: OrderIntent) -> dict[str, Any]
to_modify_route(route_id: str, changes: dict) -> dict[str, Any]
to_cancel_route(route_id: str) -> dict[str, Any]
from_create_response(response: dict, intent: OrderIntent) -> ExecutionAck
from_route_subscription(msg: dict) -> ExecutionUpdate
from_fill_message(msg: dict) -> FillUpdate
```

Status mapping

```
{SENT      → SENT,WORKING      → WORKING,PARTFILLED      →
PART_FILLED,FILLED → FILLED,CANCEL → CANCELLED,REJECTED →
REJECTED,ROUTE-ERR → ROUTE_ERR}.
```

9.3 execution.emsx_requests

EMSXRequests

Purpose Thin BLPAPI wrapper for the EMSX request/response service.

Signature

```
start() / stop()
create_order_and_route(fields: dict) -> dict      # CreateOrderAndRouteEx
create_order(fields: dict) -> dict                # CreateOrder
route_ex(fields: dict) -> dict                    # RouteEx
modify_route(fields: dict) -> dict                # ModifyRouteEx
cancel_route(fields: dict) -> dict                # CancelRouteEx
```

Raises **EMSXError** on session/service open failure or when **blpapi** is missing.

Note Unknown fields are logged but do not abort the request — EMSX schema evolution must not break callers.

9.4 execution.emsx_subscriptions

EMSXSubscriptions

Purpose	Subscribes to <code>//blp/emapisvc/order</code> and <code>//blp/emapisvc/route</code> ; dispatches each message to typed callbacks.
Signature	<pre>__init__(host, port, order_topic, route_topic, order_fields: list[str] None, route_fields: list[str] None) start() / stop() on_order_update(cb: Callable[[dict], None]) -> None on_route_update(cb: Callable[[dict], None]) -> None</pre>
Defaults	Order/Route fields cover <code>EMSX_SEQUENCE</code> , <code>EMSX_ROUTE_ID</code> , <code>EMSX_TICKER</code> , <code>EMSX_SIDE</code> , <code>EMSX_AMOUNT</code> , <code>EMSX_STATUS</code> , <code>EMSX_FILLED</code> , <code>EMSX_WORKING</code> , <code>EMSX_AVG_PRICE</code> , <code>EMSX_FILL_AMOUNT</code> , <code>EMSX_FILL_PRICE</code> ,
Resilience	Callback exceptions are caught and logged; the event loop survives.

9.5 execution.emsx_state_machine

EMSXStateMachine

Purpose	Validates that incoming venue-status transitions are legal. Detects out-of-order subscription messages.
Signature	<pre>current(order_id: str) -> OrderStatus seed(order_id: str, state=OrderStatus.UNKNOWN) -> None transition(order_id: str, new: OrderStatus) -> bool # False if illegal on_violation(cb: Callable[[order_id, current, attempted], None]) -> None</pre>
Rules	Transitions out of terminal states are violations. Allowed transitions encoded in <code>_ALLOWED</code> table.
Seed semantics	Adapters seed the state under <code>UNKNOWN</code> the moment a venue order id is allocated, so direct <code>NEW</code> <code>\$\to</code> <code>\$ WORKING</code> jumps (EMSX often skips <code>SENT</code>) are not flagged.

9.6 execution.emsx_adapter

EMSXExecutionAdapter

Purpose High-level adapter; composes EMSXRequests, EMSXSubscriptions, EMSXMapper, EMSXStateMachine.

Signature

```
__init__(config: EMSXConfig, auto_route: bool = True,
         requests: EMSXRequests | None = None,
         subscriptions: EMSXSubscriptions | None = None)

start() / stop()
submit_order(order: OrderIntent) -> ExecutionAck
cancel_order(order_id: str) -> ExecutionAck
modify_order(order_id: str, changes: dict) -> ExecutionAck
on_execution_update(cb) / on_fill(cb)
```

Routing modes

- `auto_route=True` → `CreateOrderAndRouteEx` (Phase 3+).
- `auto_route=False` → `CreateOrder` only; trader routes manually in EMSX (Phase 2 staging).

Subscription-id race — contract

The runner pre-registers each order under `intent.idempotency_key` (its internal id). EMSX subscriptions report `EMSX\SEQUENCE` (the venue id). The adapter maintains an internal `EMSX\SEQUENCE $to $ client\order\_id` mapping populated on `submit\_order` ack and *rewrites* every outgoing `ExecutionUpdate` / `FillUpdate` to carry the client id in `order_id`.

Subscription-id race — buffer

If a route subscription fires *before* the ack lands, the update is buffered keyed by `EMSX\SEQUENCE`. When `submit\_order` returns, the buffered updates are replayed under the now-known client id, in order. No updates are silently dropped.

9.7 execution.paper_adapter

PaperExecutionAdapter

Purpose Internal paper-fill engine. No external wire.

Signature

```
__init__(get_mark: Callable[[str], float | None],
         slippage_ticks: float = 0.0,
         tick_size_lookup: Callable[[str], float] | None = None)
```

Order-id convention

Uses `OrderIntent.idempotency_key` as the `order_id` in the ack and in every callback. The runner must pre-register the order under that same id in `WorkingOrderBook` and `OrderLifecycle` *before* calling `submit_order`, since the `SENT`/fill/`FILLED` callbacks fire synchronously inside the call.

Behaviour

- On accept: emits `SENT` update, a `FillUpdate`, then `FILLED` update.
- On reject (no mark available): emits a `REJECTED` update so any pre-registered working entry is cleared rather than left hanging.
- Fill price: `limit_price` for LMT; `mark + slippage\_ticks * tick\_size * side.sign` otherwise.
- Modifies are not supported.

10 Module storage/

10.1 storage.db

Database, get_database

Purpose	Minimal SQLite wrapper with shared lock and WAL journal. Schema initialized from <code>schema.sql</code> on construction.
Signature	<pre> Database(url: str) # only sqlite:/// supported execute(sql, params=()) -> None executemany(sql, params_list) -> None query(sql, params=()) -> list[sqlite3.Row] cursor() -> context manager close() -> None get_database(url: str None = None) -> Database # singleton helper </pre>
Raises	ValueError on non-sqlite URLs.

10.2 storage.event_log

EventLog (abstract), JsonEventLog, SqliteEventLog

Purpose	Append-only audit trail. Strategy/risk/execution decisions are persisted.
Signature	<code>append(event_type: str, event: Any, correlation_id: str None = None) -> None</code>
Encoding	Dataclasses are converted via <code>asdict</code> ; Enums use <code>.value</code> ; datetimes use ISO-8601.
Thread-safety	<code>JsonEventLog</code> uses an internal lock; safe to share across threads.

10.3 storage.repositories

PositionRepository, OrderRepository, FillRepository, RiskDecisionRepository

PositionRepository	<code>upsert(instrument, qty, avg_cost), get, all()</code> .
OrderRepository	<code>upsert(rec: OrderRecord, leaves_qty), by_idempotency_key(key)</code> for crash-restart dedupe.
FillRepository	<code>insert(fill), for_order(order_id)</code> .
RiskDecisionRepository	<code>insert(decision: RiskDecision)</code> .
idempotency_key uniqueness	Enforced by a UNIQUE constraint in <code>working_orders</code> , and by the dedicated <code>submitted_intents</code> table that backs <code>SqliteIdempotencyBackend</code> .

10.4 storage.idempotency_backend

SqliteIdempotencyBackend

Purpose	Persistent, race-safe dedupe backend for IdempotencyStore . Survives restart so a re-played bar cannot resubmit an already-accepted order; tolerates concurrent processes so two services racing on the same key see exactly one winner.
Signature	<pre> __init__(db: Database) has(key: str) -> bool add(key: str) -> None claim(key: str) -> bool # ATOMIC: True iff this call actually inserted </pre>
Implementation	<code>claim</code> executes a single “INSERT OR IGNORE INTO <code>submitted_intents</code> ” and inspects <code>cursor.rowcount</code> . SQLite serializes writes; <code>rowcount</code> is 1 on insert, 0 on the ignored conflict.
Wiring	Default backend assembled by <code>build_services</code> .
Schema	<code>submitted_intents</code> (<code>idempotency_key</code> PRIMARY KEY, <code>submitted_at</code> TEXT NOT NULL).

10.5 storage.snapshots

SnapshotStore

Signature	<pre> save(name: str, payload: dict) -> None latest(name: str) -> dict None </pre>
Use	Periodic blob snapshots of risk state, strategy state, etc., for warm restarts.

11 Module messaging/

11.1 messaging.bus

EventBus (abstract)

Purpose Pub/sub interface used by every service.

Signature

```
class EventBus(ABC):
    def publish(self, topic: str, payload: Any) -> None
    def subscribe(self, topic: str, handler: Handler) -> None
    def start(self) -> None
    def stop(self) -> None
```

Handler Callable[[topic: str, payload: Any], None].

Payload contract

Handlers always receive the canonical event dataclass registered in `messaging.codec` for that topic, never a raw dict — including for serializing buses.

11.2 messaging.codec

Topic registry + dataclass codec

Purpose Bridges JSON-serializing buses (Kafka, Redis) and typed handlers. Reconstructs the registered dataclass for each topic on decode so strategies and runners never have to branch on dict vs. dataclass.

Signature

```
register_topic(topic: str, event_type: type) -> None
topic_type(topic: str) -> type | None
encode(payload: Any) -> str
decode(topic: str, raw: Any) -> Any
to_jsonable(o: Any) -> Any
from_dict(cls: type, data: dict) -> Any
```

Coercion rules

Enums via `cls(value)`; `datetime` via `fromisoformat`; nested dataclasses recurse; `Optional/|None` unions strip `NoneType`; lists/tuples recurse into their element type.

Registration `app.topics` performs the canonical bindings at import: `TICKS$` to `$MarketTick`, `BARS$` to `$BarClosed`, `TARGETS$` to `$TargetPosition`, `INTENTS$` to `$OrderIntent`, `RISK\_DECISIONS$` to `$RiskDecision`, `EXECUTION\_UPDATES$` to `$ExecutionUpdate`, `FILLS$` to `$FillUpdate`, `KILL\_SWITCH$` to `$KillSwitchEvent`. `register_topic` is idempotent; rebinding a topic to a different class raises `ValueError`.

Unregistered topics

`decode` returns the raw value unchanged — safe pass-through for custom topics.

11.3 messaging.in_memory_bus

InMemoryBus

Semantics Synchronous. `publish` invokes all subscribed handlers inline.

Use Single-process deployments and tests.

Failures Handler exceptions are logged; never propagate to the publisher.

11.4 messaging.kafka_bus

KafkaBus

Signature	<code>KafkaBus(bootstrap: str, group_id: str = "fes")</code>
Encoding	<code>publish</code> runs <code>codec.encode</code> ; the consumer thread runs <code>codec.decode(topic, raw)</code> so registered topics deliver typed dataclasses.
Use	Multi-service deployments.
Optional dep	<code>confluent-kafka</code> .

11.5 messaging.redis_bus

RedisStreamBus

Signature	<code>RedisStreamBus(url: str, group: str = "fes", consumer: str = "fes-1")</code>
Semantics	Consumer-group reads via <code>XREADGROUP</code> ; <code>XACK</code> after handler success. <code>publish</code> runs <code>codec.encode</code> ; the consumer thread runs <code>codec.decode(topic, raw)</code> before invoking handlers.
Optional dep	<code>redis</code> .

make_bus (factory)

Signature	<code>make_bus(kind: str, url: str None = None) -> EventBus</code>
kind	"memory", "kafka" (requires url), "redis" (requires url).
Raises	<code>ValueError</code> for unknown kind or missing URL.

12 Module monitoring/

12.1 monitoring.metrics

Metrics

Purpose	Prometheus counters/gauges/histograms exposed on the HTTP endpoint.
Signature	<code>Metrics(port: int None = 9100)</code> (pass <code>None</code> or <code>0</code> to disable)
Per-instance registry	Each Metrics owns a dedicated CollectorRegistry , so tests, dev REPLs, and warm-restart flows can construct multiple service graphs in one process without colliding on the prometheus global registry.
Counters	<code>ticks_total</code> , <code>bars_total</code> , <code>signals_total</code> , <code>orders_submitted</code> , <code>orders_rejected</code> , <code>fills_total</code> .
Gauges	<code>open_position</code> , <code>working_qty</code> , <code>realized_pnl</code> , <code>unrealized_pnl</code> , <code>kill_switch_tripped</code> , <code>market_data_age_seconds</code> .
Histogram	<code>bar_to_order_seconds</code> (buckets 5ms ... 5s).

12.2 monitoring.health

HealthProbe, HealthChecker

HealthProbe	<code>name</code> , <code>check: Callable[[], bool]</code> , <code>detail: Callable[[], str]</code> .
HealthChecker	<pre> register(probe: HealthProbe) -> None check_all() -> dict[name -> {ok: bool, detail: str}] is_healthy() -> bool </pre>

12.3 monitoring.alerts

AlertSink, ConsoleAlertSink

Signature	<code>alert(severity: str, message: str, **fields) -> None</code>
Severity	<code>critical</code> , <code>error</code> , <code>warning</code> , <code>info</code> .
Implementations	ConsoleAlertSink (default). <code>Slack/PagerDuty</code> subclasses pluggable.

13 Module backtest/

13.1 backtest.slippage

SlippageModel (abstract), FixedTickSlippage

Signature `adjust(mid: float, side: Side, instrument: str) -> float`

FixedTickSlippage

`__init__(ticks: float, tick_size_lookup); mid + ticks*tick_size*side.sign.`

13.2 backtest.commission

CommissionModel (abstract), FlatCommissionModel

Signature `per_fill(qty: int, price: float, instrument: str) -> float`

FlatCommissionModel

`__init__(per_contract: float = 1.25); |qty| * per_contract.`

13.3 backtest.replay

BarReplay, TickReplay

BarReplay `__iter__()` sorted by `end_time`; `between(start, end)` yields a subrange.

TickReplay `__iter__()` sorted by `exchange_timestamp` or `receive_timestamp`.

13.4 backtest.engine

BacktestEngine

Purpose Drives the same **Strategy** + **OrderManager** + **PositionBook** components used live. Fills are simulated.

Signature

```
__init__(strategy: Strategy, instruments: InstrumentsConfig,
         slippage: SlippageModel | None = None,
         commission: CommissionModel | None = None)

run(bars: Iterable[BarClosed]) -> BacktestResult
```

BacktestResult `bars_processed, targets_emitted, intents_emitted, fills_count, realized_pnl, unrealized_pnl, total_pnl, total_commission, end_positions: dict[str,int], timeline: list[(datetime, PnLSnapshot)].`

Guarantee Live $\equiv$ backtest at the strategy + risk + order layer (same code paths).

13.5 backtest.reports

generate_report

Signature `generate_report(result: BacktestResult) -> dict[str, Any]`

Keys returned

`bars, targets, intents, fills, realized_pnl, unrealized_pnl, total_pnl,`
`total_commission,`
`net_pnl, sharpe_annualized, max_drawdown, end_positions.`

Sharpe convention

Per-bar returns scaled by $\sqrt{252 \cdot 390}$ minutes/year.

14 Module app/

14.1 app.topics

Topic name constants

Constants	TICKS, BARS, TARGETS, INTENTS, RISK_DECISIONS, EXECUTION_UPDATES, FILLS, KILL_SWITCH.
Rule	All services use <i>only</i> these constants when publishing/subscribing.

14.2 app.wiring

Services, build_services

Purpose	Single composition root. Every entry point builds the same DI graph.
Signature	

```
build_services(
    config_dir: str | Path, *,
    metrics_port: int | None = None,
    market_data: MarketDataProvider | None = None,
    execution: ExecutionAdapter | None = None,
) -> Services
```

Services fields	config, bus, db, event_log, metrics, health, alerts, market_data, bar_builder, tick_store, stale_monitor, positions, fills, pnl, exposure, working, idempotency, order_manager, kill_switch, risk, execution, strategies, position_repo, order_repo, fill_repo, risk_decision_repo.
-----------------	---

Warm restart	On construction, <code>position_repo.all()</code> is loaded into the in-memory PositionBook . Runners persist on every fill (<code>position_repo.upsert + fill_repo.insert</code>) and on every risk decision (<code>risk_decision_repo.insert</code>); a fresh process picks up where the previous one left off without manual reconciliation.
--------------	--

Defaults	MockMarketDataProvider and PaperExecutionAdapter when not overridden — dev/test/paper trading work out of the box.
----------	--

Idempotency wiring	IdempotencyStore is constructed with a SqliteIdempotencyBackend over the configured DB, so claimed keys persist across restarts.
--------------------	--

Metrics-port resolution	If a CLI <code>--metrics-port</code> is given it wins; otherwise <code>environments.metrics_port</code> is used. A value of 0 or less disables the Prometheus HTTP endpoint in either source.
-------------------------	---

14.3 app.run_market_data

Entry point: run_market_data

CLI	<code>python -m futures_emsx_strategy.app.run_market_data --config-dir config [--bloomberg --mock] [--metrics-port N].</code>
Behaviour	- Subscribes to instruments listed in <code>instruments.yaml</code>. - For each tick: updates InMemoryTickStore, StaleDataMonitor, MinuteBarBuilder, then publishes TICKS; BARS on close. - Persists MarketTick and BarClosed to the event log.

14.4 app.run_strategy

Entry point: `run\_strategy`

CLI	<code>python -m futures_emsx_strategy.app.run_strategy --config-dir config.</code>
Behaviour	Subscribes to BARS; routes each bar to all strategies bound to that instrument; publishes TARGETS for each <code>TargetPosition</code> emitted.

14.5 app.run_execution

Entry point: `run\_execution`

CLI	<code>... --config-dir config [--emsx --paper] [--auto-route --stage-only] [--metrics-port N].</code>
Subscriptions	TICKS (feed <code>StaleDataMonitor</code> and <code>InMemoryTickStore</code>) and TARGETS. The tick subscription is mandatory — without it the risk gate rejects every order for stale data in split deployments.
Pipeline (per TARGETS message)	1. <code>OrderManager.on_target</code> → intents. 2. Publish INTENTS; run <code>PreTradeRiskGateway.validate</code>; persist via <code>RiskDecisionRepository</code>; publish RISK_DECISIONS. 3. If approved: <code>idempotency.claim(key)</code>. Skip if it returns <code>False</code> (already submitted, e.g. duplicate target on the bus). 4. Pre-register the order in <code>WorkingOrderBook</code> and <code>OrderLifecycle</code> under <code>intent.idempotency_key</code> (status <code>NEW</code>). Then call <code>execution.submit_order</code>. 5. Use <code>OrderLifecycle.set_venue_info</code> to record <code>ack.order_id</code> / <code>ack.route_id</code>. Do not overwrite status from the ack — callbacks own it. 6. On rejection ack: write <code>REJECTED</code> status to lifecycle and zero out the working entry. 7. Inbound <code>ExecutionUpdate/FillUpdate</code> use <code>OrderLifecycle.resolve</code> and update lifecycle, working book, positions, PnL, metrics; publish <code>EXECUTION_UPDATES</code>, <code>FILLS</code>; on fill, also write to <code>FillRepository</code> and upsert <code>PositionRepository</code>.

Why pre-register?

The paper adapter fires `SENT`/fill/`FILLED` callbacks synchronously inside `submit_order`. Without pre-registration, the runner's post-submit bookkeeping would race the callbacks and leave a phantom working entry stuck at `SENT`.

Why claim post-risk, not in OrderManager?

A transient risk rejection (stale data, session) must remain retryable. Claiming after risk approves leaves the key un-committed when risk rejects, so the next bar / restart can succeed.

14.6 app.run_reconciler

Entry point: `run\_reconciler`

CLI	<code>... --config-dir config --external-positions broker.json --interval-seconds 60.</code>
Behaviour	Periodically loads <code>\{symbol: qty\}</code> from the file and compares against <code>PositionBook</code> . Emits warning alerts on breaks. Never auto-resyncs.

14.7 app.run_all_local

Entry point: `run\_all\_local`

CLI ... `--config-dir config [--emit-ticks|--no-emit-ticks] [--seconds N]`.

Behaviour One-process composition over **InMemoryBus**. Used for paper trading, dev, integration testing.
With `--emit-ticks`, drives **MockMarketDataProvider** with a synthetic random walk.

15 Topology & guarantees

15.1 Service boundaries

Service	Subscribes to	Publishes
market-data	(Bloomberg, OS)	TICKS, BARS
strategy	TICKS, BARS	TARGETS
execution	TICKS, TARGETS	INTENTS, RISK_DECISIONS, EXECUTION_UPDATES, FILLS
reconciler	(filesystem, OS)	alerts
monitoring	(Prometheus scrape)	—

Tick subscription is mandatory for any service that runs the pre-trade risk gate. Without it, **StaleDataMonitor** sees the instrument as never seen and rejects every intent for stale data.

15.2 Idempotency & at-least-once delivery

The bus is treated as *at-least-once*. Idempotency is enforced inside the **OrderManager** via `idempotency\_key`:

```
key = SHA1(strategy_id | instrument | bar_end | side | qty)[:16]
```

If the same key has already been claimed by **IdempotencyStore**, the intent is suppressed. `build_services` wires the store to **SqliteIdempotencyBackend**, persisting every claimed key into the `submitted_intents` table; a process crash or bar replay after restart therefore cannot resubmit a previously accepted order. The same key is the internal `order_id` used by paper-adapter callbacks, and is stored in `working\_orders.idempotency\_key` (UNIQUE) for the lifecycle audit trail.

15.3 State-machine invariants

EMSXStateMachine only permits the transitions in `\_ALLOWED`. Notable rules:

- Terminal states (FILLED, CANCELLED, REJECTED, ROUTE_ERR) are sinks. Any further status change triggers a violation callback.
- NEW cannot jump directly to SENT; it must first be RISK_APPROVED.
- UNKNOWN is a recovery state — any transition back into known states is allowed, used after gap-fill on restart.

15.4 Phased rollout flags

Phase	Adapter	Notes
1	PaperExecutionAdapter	No external wire; full audit
2	EMSXExecutionAdapter <code>auto_route=False</code>	Stages orders; trader routes
3	EMSXExecutionAdapter <code>auto_route=True</code>	Tight <code>risk_limits.yaml</code> , one symbol, small size
4	Same as 3	Production limits, reconciler + kill switch live

15.5 Error budget for live trading

Any of the following must trip the kill switch (manually or via alert action):

- Market data age > `stale_data_seconds` (default 30) for any subscribed instrument.
- Sustained risk rejections (e.g., > 1/s for 5 min).
- State-machine violations (`EMSXStateMachine.on\_violation`).
- Position breaks reported by **Reconciler**.

16 Quick reference — minimum contracts to satisfy when extending

16.1 Adding a market-data vendor

Implement **MarketDataProvider** (see `market_data.base`). Bind it into `build_services` via the `market\_data` argument. No other layer changes.

16.2 Adding a venue

Implement **ExecutionAdapter**. Provide a venue-specific mapper analogous to **EMSXMapper**. Reuse **EMSXStateMachine** or write a venue-specific one. Bind via the `execution` argument to `build_services`.

16.3 Adding a strategy

Subclass **Strategy**. Read only via **PortfolioView**. Emit **TargetPosition**. Add an entry under `strategies.yaml`; the wiring layer instantiates it.

16.4 Adding a risk check

Add a private method `_check_*` to **PreTradeRiskGateway** that returns a **ValidationCheck**. Append it to the list inside `validate`. Tests must show that a failing case rejects the order with that check's name in `reasons`.

16.5 Adding a bus implementation

Implement **EventBus**. Register it in `make_bus`. The new transport must:

- Serialize dataclasses correctly (see **KafkaBus** pattern).
- Catch and log handler exceptions.
- Provide at-least-once delivery (so **IdempotencyStore** can dedupe).

17 Test contract surface

Every test in `futures\emsx\strategy/tests/` corresponds to a published invariant.

Test	Invariant being enforced
<code>unit/test_bar_builder.py</code>	Bars close only on a strictly later minute; <code>flush()</code> drains pending.
<code>unit/test_order_manager.py</code>	Δ math, working-qty offset, sign of <code>Side.from_delta</code> , idempotency dedupe.
<code>unit/test_pre_trade_risk.py</code>	Each risk check rejects exactly its scenario; happy path approves.
<code>unit/test_positions_pnl.py</code>	Weighted-average cost, realized PnL with point value, unrealized when flat.
<code>unit/test_positions_avg_cost.py</code>	Partial close keeps avg; full close resets to 0; flip sets avg to fill price.
<code>unit/test_codec.py</code>	Topic round-trip preserves enums and <code>datetime</code> ; unregistered topics pass through.
<code>unit/test_paper_cleanup.py</code>	Paper fills leave no phantom working orders; rejected ack clears the pre-register.
<code>unit/test_sqlite_idempotency.py</code>	SQLite-backed claims survive a fresh Database instance on the same file.
<code>unit/test_bar_invariants.py</code>	Historical bars satisfy <code>end_time > start_time</code> from any provider.
<code>unit/test_emsx_race.py</code>	EMSX route updates arriving before <code>submit_order</code> ack are buffered and replayed; all updates resolve to the client id.
<code>unit/test_idempotency_atomic.py</code>	Concurrent claim on the same key yields exactly one winner (SQLite + in-memory).
<code>unit/test_risk_rate_limit_deferred.py</code>	Rejected intents do not drain the rate limiter; approved intents consume one slot each.
<code>unit/test_synthetic_tick_clock.py</code>	Synthetic tick generator's timestamps are strictly monotonic across hour and day boundaries.
<code>unit/test_bloomberg_provider_config.py</code>	Bloomberg provider applies <code>topic_to_symbol</code> ; honors host/service args.
<code>integration/test_state_restore.py</code>	Positions persist on fill and are restored on a fresh <code>build_services</code> .
<code>integration/test_split_service.py</code>	Execution-service tick subscription feeds StaleDataMonitor .
<code>unit/test_state_machine.py</code>	Legal transition chain; post-terminal transitions are violations.
<code>unit/test_indicators.py</code>	Indicators return <code>None</code> until ready; constant series $\Rightarrow$ z-score 0.
<code>integration/test_end_to_end_paperfills.py</code>	Ticks $\rightarrow$ bars $\rightarrow$ strategy $\rightarrow$ risk $\rightarrow$ paper fills $\rightarrow$ position update.
<code>integration/test_backtest.py</code>	Backtest engine runs and reports keys exist.
<code>emsx_sim/test_emsx_adapter_fake.py</code>	Adapter pipeline against an in-process BLPAPI fake; route subscription $\rightarrow$ fill propagation.
<code>replay/test_replay_deterministic.py</code>	Same bar series $\Rightarrow$ identical orders + idempotency keys.

End of document.